

(the)

UPSET

QUARTET

Ivan, Tahmid, Tyler and Ethan click here!

Project Manager: Ivan

TNPG: Upset Quartet

SoftDev

P00: Half-Quick

TARGET SHIP DATE: 2026-11-20 (TBD)

Time Spent: 2.5 hrs

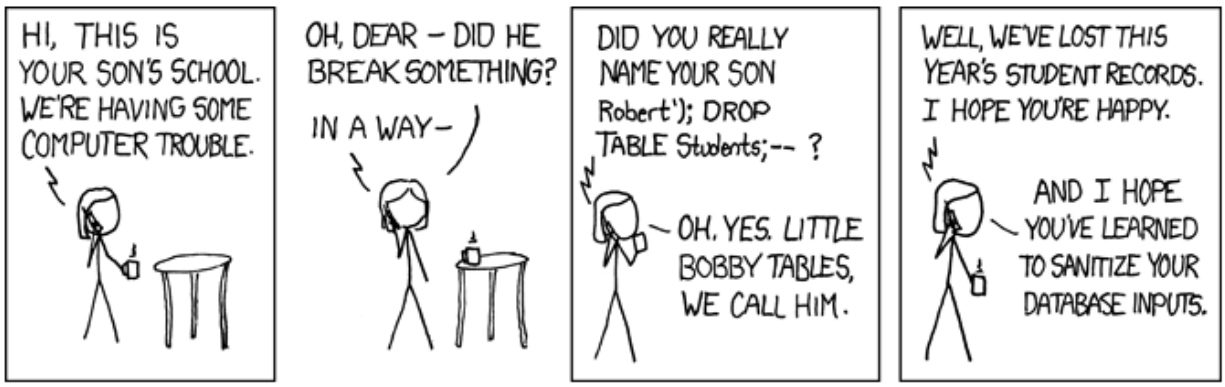
Make sure to claim your role at the very bottom!

List of Program Components

and Site Map because both were done at the same time!

- There are the necessary and optional pages for this project to work.
    - main.html : Main page that the user is redirected to, regardless of login status. Please note that unlike the previous project, the user does **not** have to be logged in to view data.
    - login.html : Login page for the user to log into their account. This may also be incorporated as some sort of widget in each page, to facilitate user log in without having to redirect the user to this page, but it's harder to incorporate. If the user is logged into an account, they should not be able to access this page.
    - logout.html : Logout page that doesn't have content inside it. It should automatically redirect the user to the main page.
    - signup.html : Sign up page for the user to sign up into their account. If the user is already logged into their account, they should not be able to access this page.
    - create.html : This is the page that the signed-in user goes to create a page on the wiki. As a further note, main.html and other html pages except for, for example, login, signup and create should be editable.
    - /page/[title].html : Every page that a user creates should be in this format.
    - /templates/[title].html : This is optional as a feature given enough time. The template page should be copyable, serving as the basis for each page.

    - /user/[username].html : This is a profile page that each user should be able to see. Each user should only be able to edit their own page, but can view any other regardless of login status.
    - /edit/[page].html : This is a page where you get redirected to once you request to edit a page. Keep in mind that the edit page *itself* is not editable, but the content inside of it is.
    - index.html : This is an optional page given enough time. The user shouldn't have to spend time going between hyperlinks to find a specific page, but to query the SQLite table to find the page they need.
- As a functional member of society, humans prefer aesthetics. Although raw HTML webpages are satisfactory, they might not cut the chase. Therefore, similar to sites like Wikipedia, there needs to be both Markdown (and possibly rich text) and CSS processing.
- Furthermore, as functional members of societies, assume each human to be an incompetent as possible. In other words, try to break the site as much as you can: make sure authenticated users can do authenticated things and non-authenticated users can't. What would happen in somebody named themselves
  > <script>scriptInjection()</script>https://www.google.com/admin/null
  or something like that?



What not to do. It's called an SQL injection for a reason!

- There are other boring things, but it's pretty self-evident. There needs to be a Flask server running — because of our limited knowledge — in order to get the website up. Unlike

concatenating user information into an easily accessible text file, it's now stored into a SQLite3 table. And finally, Jinja for your favorite usage of {{ variable }}.

Possible Routing:

Logged-Out User

main.html -> index.html -> /page/page.html -> /edit/[page].html (cannot access this website, redirected back to…) -> main.html -> login.html (does not log in) -> main.html -> signup.html (does not sign up) -> main.html -> create.html (cannot access this website, redirected back to…) -> main.html

Logged-In User

main.html -> /user/[username].html -> main.html -> create.html (creates page) -> /page/[title].html -> /edit/[page].html -> logout.html (logs out) -> main.html

Component Relationships

Also known as the client-server interactions… This'll be tough.
It'll be easier to explain through several different scenarios: what will happen at each step?

Scenario I: The user creates a web page

1) On the local client, the user hits the submit button. This sends a POST request to the Flask web server.
2) Flask checks the client's information, primarily by their cookie, to see if they are logged in. If not, then Flask tries to redirect the user to the main page. Otherwise, Flask requests the necessary information from a SQLite3 table, like page title. However, since the user is creating a page, this step isn't needed unlike when the user is updating the page.
3) The table should return some data back to Flask. Thereafter, it would render templates using Jinja, compile the HTML, attach the necessary CSS and zip that up ready for the user to read.

Keep in mind that this is an ultra-simplified version of what goes on. Again, Robert"); DROP TABLE Students;--. There's not enough room here, go down to view the component diagram.

Scenario II: The user creates an account.

1) On the local client, the user hits the submit button. This sends a POST request to the Flask web server.
2) Flask checks the client's information, primarily by their cookie, to see if they are logged in. If so, then Flask tries to redirect the user to the main page. Otherwise, Flask requests the necessary information from a SQLite3 table, to further check if the account creation is valid. If the username is already taken, a field isn't filled out, or there are no matching passwords, then an error message is displayed. Then, the Flask server should append the user information to the necessary table.
3) Once the information has been safely stored, Flask then needs to create a cookie, logging the user into their new account. The user should be redirected to the main page, happily logged in.

Now, what would happen if two people tried to create the same account at the same time using different passwords?

Scenario III: The user edits an existing page.

For the sake of simplicity, a note about Flask/SQLite3 tables will merely be written instead of restating the same thing over and over again. This time, Flask needs to read the data associated with the current page and modify/replace the data.

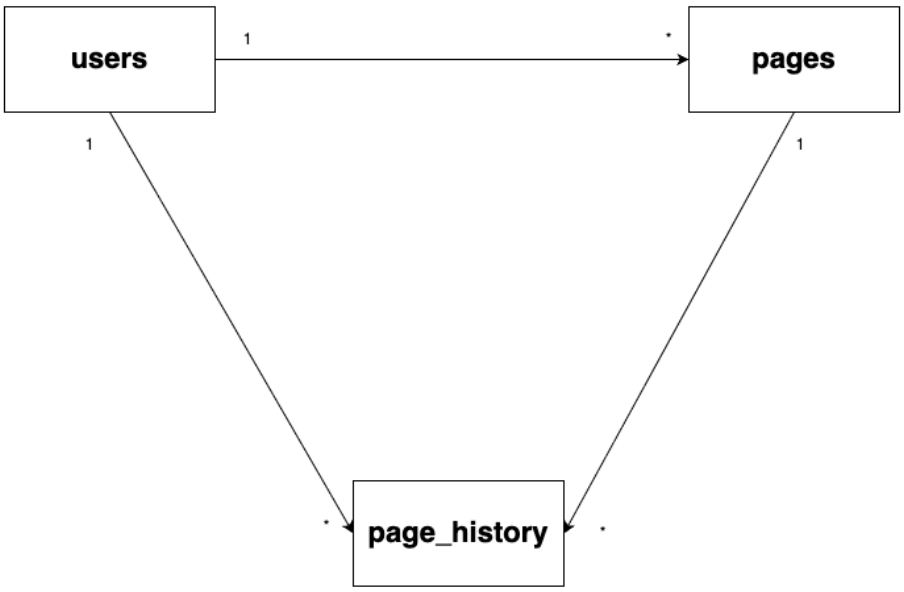
Scenario IV: The user searches for every page in the index.

Beyond reading every page in the SQLite3 table and storing it in a list to be displayed, if the user tries to search for a certain character ("a"), what results should be listed foremost? Should there be customizable ways to sort the pages you read?

In summary, most of the actions that take place involve five key steps.
The user requests an operation to be handled.
Flask confirms that the user has the capability to do so, and if necessary, in multiple ways.
Flask requests/modifies/appends/deletes information from the SQLite3 table.
Flask modifies/redirects/creates a web page with the necessary information for the viewer.
The user is able to see the changes made once the operation has been completed.

Database Organization

There needs to be three tables that are necessary for the advent of this production. The first table necessary is the users table, that has the columns id: integer (technically each username is unique, but the id is a stable way of verifying the authenticity and uniqueness of each user; this can also be used in place of the username as the appendix of a link), username: String, password: String. I'm sure that if needed, the password could be stored in a secure manner like base64, but cybersecurity is not a concern of ours at this very moment. The second table necessary is the pages table, where each individual page is stored with the raw content. It should have an id: integer, title: String, text: String, creator_id: integer, and created_at: integer. Some context needed is that Unix timestamps can be stored as an integer, and that last_editor_id and last_updated both rely on the next table: page_history. This should run in conjunction with the pages table, storing information like id: integer (the same as page's id), editors: list (list of last_editor_id*s*), timestamps: list (list of last_updated*s*). As a further note, if you need to revert a change to a website, will there need to be content stored in the page_history table so that an editor can easily revert back? What would be the best way to store changes, then?
P.S: After further research, it isn't possible to store lists as an item inside a SQLite3 table, unless you are willing to work large amounts of workarounds. Therefore, the id should be separated into id and page_id; each table should therefore be replaced with last_editor_id and last_updated. Thereby, if you want to get the information of the actual last editor, you would find the page's id, you would call every row in the table and get the one with the latest (largest) last_updated.



A simple diagram showing the cardinality of each table.

Please do not fret over whether the symbol should be an asterisk or the letter N.

Task Breakdown and Assignments of Each Task

As for right now, I'm going to be using generic placeholder names. Please **claim** each role you want to be; if each person utilizes their strengths, there shall be no infighting!

Person 1: Lin, Tyler
SQLite3 Database Architect
manage the tricky tables!

Person 2: Uddin, Tahmid
Authentication and Session Master
make sure no user can break the scripts!

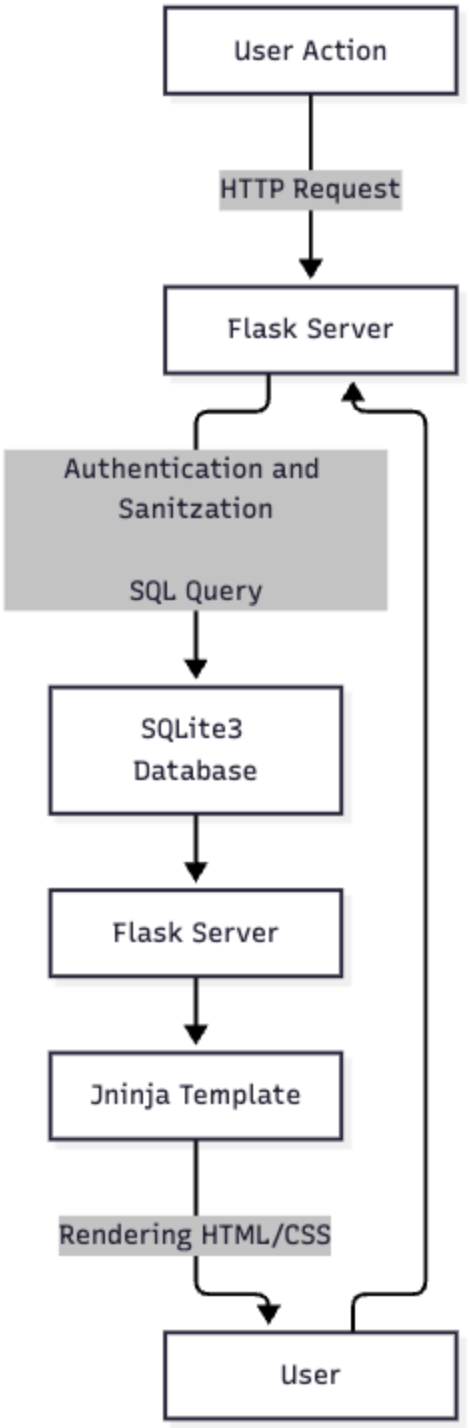
Person 3: Chan, Ivan
Wiki Engine and Page Handler
writes the core engine utilized!

Person 4: Chang, Ethan
Documentation and Front-End Developer
maintains good readmes and page customization!

I'm not bothered to write each responsibility out.
You can figure it out yourself.
Please keep in mind that if you are done with your work, please help others!

Component Relationship Diagram (cont.)

I totally didn't forget to do this and had to append this to the end, nosiiree.
See next page for diagram.



slight correction: it's Jinja, not Jninja.